

Cryptosystème Hybride adaptatif AES-ENSA

Encadré par : Pr. Fatima SENNOUNI

Préparé par :

DIALLO Abdoul-Moumouni
KUNAKA Daniel
SIMPORE Taobata

17 avril 2026

Table des matières

1 Introduction et Contexte **2**

2 Phase 1 : Le Coeur Symétrique : Variante AES-ENSA **2**
2.1 Présentation de l'AES standard . . . 2
2.2 La S-Box personnalisée AES-ENSA . . . 2
2.2.1 Définition mathématique . . . 2
2.2.2 Condition de bijectivité et correction de a . . . 2
2.2.3 Extrait de la S-Box générée ($a = 3$, $b = 2$) . . . 3
2.3 Implémentation des transformations . . . 3
2.3.1 ShiftRows . . . 3
2.3.2 MixColumns . . . 3
2.3.3 KeyExpansion . . . 3

3 Analyse de Sécurité : Vulnérabilité de la S-Box Affine **4**
3.1 Rappel : le rôle de la Confusion (Shannon, 1949) . . . 4
3.2 La S-Box de l'AES standard : l'inversion dans $GF(2^8)$ . . . 4
3.3 Vulnérabilité de la fonction affine $f(x) = ax + b$ . . . 4
3.3.1 Justification mathématique : attaque par corrélation linéaire . . . 4
3.3.2 Comparaison des propriétés cryptographiques . . . 5

4 Phase 2 : Architecture Hybride **5**
4.1 Choix des paramètres RSA : pourquoi $n > 2^{128}$ . . . 5
4.2 Structure du fichier chiffré . . . 5
4.3 Mode CBC : Cipher Block Chaining . . . 6
4.4 Padding PKCS#7 . . . 6
4.5 Gestion des erreurs : clé RSA incorrecte . . . 6

5 Analyse de l'Intégrité : Rejet HMAC **6**
5.1 Principe du HMAC-SHA256 . . . 6
5.2 Simulation de rejet : modification d'un octet du Payload . . . 7

6 Test d'Effet d'Avalanche **7**
6.1 Définition : Principe de Diffusion de Shannon . . . 7
6.2 Protocole et Résultats . . . 7

7 Synthèse et Conclusion **8**

1 Introduction et Contexte

Ce rapport présente la conception et l'implémentation de **VaultGCSE2**, un outil de stockage sécurisé à architecture *hybride*. L'objectif est de chiffrer des fichiers binaires quelconques (images, PDF, textes) en combinant :

- Le **chiffrement symétrique** (AES-ENSA-128 en mode CBC) pour la performance sur les données volumineuses.
- Le **chiffrement asymétrique** (RSA, $n > 2^{128}$) pour la protection de la clé AES.
- Le **HMAC-SHA256** pour garantir l'intégrité du fichier chiffré.

Rappel du cours : La cryptographie symétrique est extrêmement rapide mais nécessite un canal sécurisé pour échanger la clé K. C'est précisément là qu'intervient RSA : $E_K(M) = C$ et $D_K(C) = M$, avec K protégée par la clé publique RSA de l'administrateur.

2 Phase 1 : Le Coeur Symétrique : Variante AES-ENSA

2.1 Présentation de l'AES standard

L'AES (*Advanced Encryption Standard*), adopté par le NIST en 2001, est un chiffrement par bloc opérant sur une matrice d'état de 4×4 octets (128 bits). Pour une clé de 128 bits, il effectue **10 tours** (rounds), chacun composé de quatre transformations :

1. **SubBytes** : Substitution non-linéaire via la S-Box (Confusion).
2. **ShiftRows** : Décalage cyclique des lignes (Diffusion horizontale).
3. **MixColumns** : Multiplication matricielle dans $GF(2^8)$ (Diffusion verticale).
4. **AddRoundKey** : XOR bit à bit avec la sous-clé du tour.

2.2 La S-Box personnalisée AES-ENSA

2.2.1 Définition mathématique

Au lieu de la S-Box standard de Rijndael (basée sur l'inversion dans $GF(2^8)$), la variante **AES-ENSA** utilise une **permutation affine** :

$$\boxed{f(x) = (a \cdot x + b) \bmod 256}$$

Nos numéros matricules, nous membres du groupe, à l'ENSA sont de la forme ENSATE22.. On considère ici que deux premiers chiffres du matricule étudiant sont 2 et 2, donc $a = 2$ et $b = 2$. Cependant, nous montrerons ci-dessous que $a = 2$ ne satisfait pas la condition de bijectivité et doit être corrigé.

2.2.2 Condition de bijectivité et correction de a

Pour que la S-Box soit une **permutation** (indispensable pour le déchiffrement), f doit être bijective sur $\mathbb{Z}_{256}$. Cela exige que a soit **inversible modulo 256** :

$$\text{pgcd}(a,\,256) = 1 \iff a \text{ est impair}$$

Problème : $a = 2$ est pair, donc $\text{pgcd}(2, 256) = 2 \neq 1$. La fonction $f(x) = 2x + 2$ n'est pas injective : par exemple $f(0) = 2$ et $f(128) = 258 \bmod 256 = 2$. La S-Box ne couvre que 128 valeurs distinctes, ce qui la rend inutilisable pour le déchiffrement.

Correction : On force a à l'impair le plus proche en appliquant $a \leftarrow a \,|\, 1$ (mise à 1 du bit de poids faible) :

$$a = 2 \xrightarrow{a\,|\,1} a = 3, \qquad \text{pgcd}(3,\,256) = 1 \checkmark$$

Les paramètres retenus sont donc $\boxed{a = 3,\ b = 2}$. La S-Box inverse est : $f^{-1}(y) = 171\,(y - 2) \bmod 256$ car $3 \times 171 \equiv 1 \pmod{256}$.

2.2.3 Extrait de la S-Box générée ($a = 3$, $b = 2$)

ligne \ col	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x	02	05	08	0B	0E	11	14	17	1A	1D	20	23	26	29	2C	2F
1x	32	35	38	3B	3E	41	44	47	4A	4D	50	53	56	59	5C	5F
2x	62	65	68	6B	6E	71	74	77	7A	7D	80	83	86	89	8C	8F

(13 lignes supplémentaires voir `trace_test.txt`)

2.3 Implémentation des transformations

2.3.1 ShiftRows

La ligne r est décalée cycliquement de r positions vers la gauche :

$$\begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix} \xrightarrow{\text{ShiftRows}} \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{pmatrix}$$

2.3.2 MixColumns

Chaque colonne $(s_0, s_1, s_2, s_3)^T$ est multipliée dans $GF(2^8)$ par :

$$\begin{pmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{pmatrix} \begin{pmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{pmatrix}$$

La multiplication par 2 dans $GF(2^8)$ est implémentée par décalage gauche avec réduction modulo $x^8 + x^4 + x^3 + x + 1$ (XOR avec `0x1B` si débordement).

2.3.3 KeyExpansion

À partir de la clé initiale de 16 octets, l'algorithme génère 44 mots de 4 octets (11 round-keys). Tous les 4 mots, une transformation spéciale applique `RotWord`, `SubWord` (avec notre S-Box AES-ENSA) et XOR avec $\text{Rcon}[i]$.

3 Analyse de Sécurité : Vulnérabilité de la S-Box Affine

3.1 Rappel : le rôle de la Confusion (Shannon, 1949)

D'après le cours, la **Confusion** a pour but de rendre la relation entre la clé secrète et le texte chiffré *aussi non-linéaire que possible*. Une S-Box est sécurisée si elle résiste aux attaques par **cryptanalyse linéaire** et **cryptanalyse différentielle**.

3.2 La S-Box de l'AES standard : l'inversion dans $GF(2^8)$

La S-Box de Rijndael est construite par :

1. L'inverse multiplicatif $x \mapsto x^{-1}$ dans $GF(2^8)$ ($0^{-1} = 0$).
2. Une transformation affine dans $GF(2)^8$.

L'inversion dans $GF(2^8)$ est une opération de **degré algébrique 7**, hautement non-linéaire : son meilleur approximant linéaire a un biais de 2^{-4} seulement, ce qui lui confère une excellente résistance à la cryptanalyse linéaire.

3.3 Vulnérabilité de la fonction affine $f(x) = ax + b$

Vulnérabilité critique : Linéarité totale

La fonction $f(x) = ax + b \pmod{256}$ est affine (degré 1), donc linéaire. Pour tout $x, x' \in \mathbb{Z}_{256}$:

$$f(x) \oplus f(x') = (ax + b) \oplus (ax' + b) = a \cdot (x \oplus x')$$

La différentielle de f est **constante** : $\Delta f = a \cdot \Delta x$. Un attaquant peut construire directement un système d'équations linéaires reliant texte clair, texte chiffré et clé.

3.3.1 Justification mathématique : attaque par corrélation linéaire

En cryptanalyse linéaire (Matsui, 1993), on cherche une approximation linéaire :

$$\bigoplus_{i \in A} x_i \ \oplus \ \bigoplus_{j \in B} f(x)_j = 0 \quad \text{avec probabilité } p \neq \tfrac{1}{2}$$

Pour la S-Box AES-ENSA $f(x) = 3x + 2$: puisque f est affine, toute combinaison linéaire des bits d'entrée est aussi une combinaison linéaire des bits de sortie. En représentation matricielle sur $GF(2)^8$:

$$\mathbf{y} = M_3 \cdot \mathbf{x} \oplus \mathbf{b}$$

où M_3 est la matrice de multiplication par 3 dans $GF(2)^8$. Cette relation est **exacte** ($p = 1$, biais $= \frac{1}{2}$). Un attaquant résout un système linéaire de 128 équations à 128 inconnues avec quelques dizaines de paires (clair, chiffré), au lieu des 2^{128} opérations que nécessite l'AES standard.

Pour la S-Box Rijndael : aucune approximation linéaire ne peut avoir un biais supérieur à 2^{-4}, rendant la cryptanalyse linéaire impraticable.

3.3.2 Comparaison des propriétés cryptographiques

Critère	S-Box AES-ENSA $(3x+2)$	S-Box Rijndael (x^{-1})
Degré algébrique	1 (linéaire)	7 (maximal)
Non-linéarité	0	112 (maximale)
Biais linéaire max.	1/2 (parfait)	2^{-4}
Résistance cryptanalyse linéaire	**Nulle**	**Forte**
Résistance cryptanalyse différentielle	**Faible**	**Forte**
Complexité d'attaque	Polynomiale	2^{128}

CONCLUSION

La S-Box AES-ENSA $f(x) = 3x+2$ est **cryptographiquement faible** : sa linéarité totale permet à un attaquant de retrouver la clé par résolution d'un système linéaire, avec quelques dizaines de paires connues (clair, chiffré), au lieu des 2^{128} opérations nécessaires contre l'AES standard.

4 Phase 2 : Architecture Hybride

4.1 Choix des paramètres RSA : pourquoi $n > 2^{128}$

Dans notre module RSA du mini-projet précédent, nous utilisions de petits nombres premiers p=73, q=151. Mais ici, la clé AES-128 est un entier sur 128 bits (valeur maximale $2^{128}-1$). Pour la chiffrer avec RSA, le module $n = p \cdot q$ doit satisfaire $n > 2^{128}-1$, sinon $M \geq n$ et le chiffrement RSA est impossible (non inversible).

Nous utilisons deux grands premiers :

Paramètre	Valeur
p	895012521705992052691
q	320097664377407181899
$n = p \cdot q$	286491417786621504450330042252200629440209
$\|n\|$ en bits	138 bits
$n > 2^{128}$	**Oui** ✓
e (exposant public)	65537 (standard RSA)

Toute clé AES de 128 bits est un entier $M < 2^{128} < n$: la condition $M < n$ est donc garantie pour toute clé générée aléatoirement.

4.2 Structure du fichier chiffré

HEADER : 256 octets	Clé AES chiffrée par RSA
IV : 16 octets	Vecteur d'initialisation CBC
PAYLOAD : taille variable	Données chiffrées AES-ENSA CBC
FOOTER : 32 octets	HMAC-SHA256 (Header + IV + Payload)

4.3 Mode CBC : Cipher Block Chaining

Le mode **ECB** est **interdit** : deux blocs identiques produisent deux chiffrés identiques, révélant des motifs. Le mode **CBC** résout ce problème :

$$C_i = \text{AES-ENSA}_K\big(M_i \oplus C_{i-1}\big), \qquad C_0 = IV$$

4.4 Padding PKCS#7

$$\underbrace{d_1\ d_2\ \ldots\ d_{13}}_{\text{données (13 octets)}}\ \underbrace{\texttt{03 03 03}}_{\text{padding}}$$

4.5 Gestion des erreurs : clé RSA incorrecte

Si la clé privée RSA est incorrecte, le programme détecte l'erreur à deux niveaux :

1. **HMAC invalide :** La clé AES déchiffrée avec la mauvaise clé RSA est du bruit aléatoire. Le HMAC recalculé ne correspond pas au footer → rejet immédiat avant tout déchiffrement AES.
2. **Padding PKCS#7 incohérent :** Même sans contrôle HMAC, le déchiffrement avec une mauvaise clé produirait un padding invalide (valeur > 16) → exception levée.

Propriété Encrypt-then-MAC : En vérifiant le HMAC *avant* le déchiffrement, toute modification est détectée avant d'invoquer l'AES y compris en cas de mauvaise clé RSA.

5 Analyse de l'Intégrité : Rejet HMAC

5.1 Principe du HMAC-SHA256

Le HMAC couvre Header + IV + Payload, avec la clé AES comme clé HMAC :

$$\text{HMAC-SHA256}(K, m) = H\big((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m)\big)$$

5.2 Simulation de rejet : modification d'un octet du Payload

Nous avons chiffré un document, puis modifié l'octet 4 du Payload (`XOR 0xFF`) avant de tenter le déchiffrement. Voici la sortie réelle du programme :

```
======================================================
  [DECRYPT] document_test_corrompu.vault  ->  jamais_cree.txt
======================================================
  Taille du fichier  : 384 octets
  Header extrait     : 256 octets
  IV extrait         : 99C3BFCE37192BF6480B00EE7250BE15
  Payload extrait    : 80 octets
  Footer (HMAC)      : C018FE249E5645765AE9D4A690AB428125C8E849A590919FDBA88D7B274579
C7

  [RSA] Dechiffrement du header...
  Cle AES recuperee  : 4CFBB02CEBA5559B26F358D11D8A136F

  [HMAC] Verification de l'integrite...
  HMAC attendu  : C018FE249E5645765AE9D4A690AB428125C8E849A590919FDBA88D7B274579C7
  HMAC calcule  : A6ED5C66E50DCAB9DCE02D2DC273E0FBEB9474D390753DA7A0AD3E25DB28CACC

  [ERREUR CRITIQUE] HMAC invalide -- fichier corrompu ou modifie !
  -> Dechiffrement interrompu. Aucune donnee exposee.
  OK REJET CORRECT
```

Les deux HMAC diffèrent totalement malgré la modification d'un seul octet, ce qui confirme la propriété de diffusion de SHA-256. Le déchiffrement AES n'est jamais appelé : aucune donnée n'est exposée.

6 Test d'Effet d'Avalanche

6.1 Définition : Principe de Diffusion de Shannon

Modifier **1 seul bit** en entrée doit statistiquement modifier **50% des bits** en sortie. C'est la propriété de **Diffusion** assurée par ShiftRows et MixColumns.

6.2 Protocole et Résultats

On génère une clé K, puis K' avec $K'[0] = K[0] \oplus \texttt{0x01}$ (1 seul bit modifié), et on compare les chiffrés C et C' :

Paramètre	Valeur
Taille du message test	124 octets
Total de bits comparés	1024 bits
Bits différents	595/1152
Pourcentage modifié	**51,65%**
Seuil requis	$> 50\%$
Conclusion	**Effet d'avalanche confirmé ✓**

7 Synthèse et Conclusion

Composant	Implémenté	Validé
S-Box affine ($a = 3$, $b = 2$ du matricule, bijectivité vérifiée)	✓	✓
SubBytes / InvSubBytes (AES-ENSA)	✓	✓
ShiftRows / InvShiftRows	✓	✓
MixColumns / InvMixColumns dans $GF(2^8)$	✓	✓
KeyExpansion (11 round-keys)	✓	✓
Mode CBC avec IV aléatoire	✓	✓
Padding PKCS#7	✓	✓
RSA avec $n > 2^{128}$ (vraie clé AES 128 bits)	✓	✓
HMAC-SHA256 (intégrité)	✓	✓
Rejet HMAC si modification du fichier	✓	✓
Test d'effet d'avalanche ($> 50\%$)	✓	✓
Fichier de trace `trace_test.txt`	✓	✓

CONCLUSION

VaultGCSE2 implémente avec succès une architecture hybride complète. La limite principale est la faiblesse de la S-Box affine ($a = 3, b = 2$), inhérente au cahier des charges : sa linéarité totale (degré algébrique 1, non-linéarité 0) expose le système aux attaques par corrélation linéaire. Pour un usage en production, la S-Box de Rijndael (inversion dans $GF(2^8)$, non-linéarité = 112) est indispensable.